

Projet Python — Séance 1

Premiers pas en Python, Git/GitHub et premier contact avec les données réelles (BCE)

Enseignante : Rahma GARGOURI — 2026-2027

Séance de 3 heures — Projet en groupe

Objectif. Installer et prendre en main les outils (Python, Git et GitHub), écrire ses premiers scripts, puis réaliser un premier appel à l'API de taux de change de la Banque centrale européenne (données réelles). **Aucune connaissance préalable de Python n'est requise.**

Après chaque petite partie, une boîte bleue rassemble des questions à **rédiger** dans `docs/notes_seance1.md` et à **commiter**. L'IA est autorisée, mais chaque réponse doit être comprise (elle pourra être demandée à l'oral).

Partie A — Premiers pas en Python

A.1 — Installer Python. Installer Python (version 3.11 ou plus récente) et vérifier l'installation en affichant sa version dans un terminal.

A.2 — Premier programme. Écrire un premier programme `hello_world.py` qui affiche un message, puis l'exécuter.

Questions — à rédiger et à commiter

1. C'est quoi Python ? À quoi sert ce langage ? Est-il compilé ou interprété ?
2. Comment installe-t-on Python ? Comment vérifie-t-on qu'il est bien installé et quelle version on utilise ?
3. Citez les différentes façons de lancer du code Python et un cas d'usage de chacune.

A.3 — Les types de base. Créer une variable de chaque type (entier, décimal, chaîne de caractères, booléen, liste, dictionnaire) et afficher son type avec `type()`.

Questions — à rédiger et à commiter

1. Quels sont les types de base en Python ? Que signifie « typage dynamique » ? Que donne `"3" + 4` et pourquoi ?

A.4 — Les opérations. Écrire un script qui **teste chacune** des opérations suivantes et affiche le résultat obtenu :

- arithmétique sur les nombres : `+` `-` `*` `/` `//` `%` `**`;
- chaînes de caractères : concaténation et quelques méthodes (`.upper()`, `.split()`...);
- listes : ajout d'un élément, longueur, accès par indice;
- dictionnaires : accès à une valeur, ajout d'une clé;
- comparaisons : `==` `!=` `<` `>` `<=` `>=`;
- opérateurs booléens : `and`, `or`, `not`.

Questions — à rédiger et à commiter

1. Quelle est la différence entre un opérateur booléen et un opérateur logique ?

A.5 — Les conditions. Exercice. On fixe une variable `taux` (par exemple `taux = 1.08`). Écrire un `if / elif / else` qui affiche :

- « au-dessus de 1 » si `taux` est strictement supérieur à 1 ;
- « égal à 1 » si `taux` vaut exactement 1 ;
- « en-dessous de 1 » sinon.

Tester le programme avec plusieurs valeurs de `taux`.

A.6 — Les boucles. Parcourir une liste avec `for` et répéter avec `while` (par exemple : parcourir une liste de taux et afficher chaque valeur, puis en calculer la somme).

Questions — à rédiger et à commiter

1. Quand utilise-t-on une boucle `for` et quand une boucle `while` ?

Partie B — Git et GitHub

B.1 — Installer Git. Installer Git et le configurer (nom, adresse e-mail).

B.2 — Créer le dépôt. Créer le dépôt GitHub **privé** `projet-python-groupe-X`. À ce stade, il ne contient qu'un `README.md` listant les **noms des étudiants du groupe**. Ajouter l'enseignante en collaborateur.

B.3 — Premier commit. Cloner le dépôt en local, créer la branche `seance-1`, faire un premier commit et le pousser (`push`).

Questions — à rédiger et à commiter

1. C'est quoi Git ? C'est quoi GitHub ? Quelle est la différence entre les deux ?
2. Pourquoi utiliser Git et GitHub (suivre l'historique, revenir en arrière, travailler à plusieurs) ?

Partie C — Découvrir les données, puis appeler l'API

C.1 — Explorer un dictionnaire (avant tout appel réseau). Une réponse de l'API est un **dictionnaire** Python. Voici un exemple de réponse (taux du dollar pour 1 euro à une date) :

```
reponse = {
    "amount": 1.0,
    "base": "EUR",
    "date": "2024-01-02",
    "rates": {"USD": 1.0956},
}
```

Sur ce dictionnaire :

1. Afficher la valeur associée à une clé (par exemple la date, puis le taux USD dans la partie `rates`).
2. Ajouter une paire clé-valeur dont la clé **existe déjà** (par exemple `reponse["date"] = "2099-01-01"`), puis afficher le dictionnaire. **Qu'est-ce qui se passe ?**

3. Copier le dictionnaire (`copie = reponse.copy()`), modifier la copie, afficher l'original : **est-il modifié** ? Recommencer en modifiant la partie imbriquée `rates` de la copie. Que constatez-vous ?
4. Refaire une copie qui protège aussi les parties imbriquées (**copie profonde**) et vérifier que l'original n'est plus affecté.

Questions — à rédiger et à commiter

1. Quand on affecte une valeur à une clé qui existe **déjà** dans un dictionnaire, que se passe-t-il ? Un dictionnaire peut-il contenir deux fois la même clé ?
2. Après avoir copié un dictionnaire avec `.copy()` et modifié la copie, l'original est-il modifié ? Et si l'on modifie une partie imbriquée (comme `rates`) ? Expliquez.
3. C'est quoi la **mutabilité** ? Quelle différence entre une copie superficielle et une copie profonde ? Comment fait-on une copie profonde en Python ?
4. Quels types de base sont **mutables** ? Lesquels sont **immuables** ? Donnez des exemples.

C.2 — Appeler l'API pour de vrai.

L'API **Frankfurter** fournit les taux de change de référence de la BCE, gratuitement et **sans clé**. Les taux sont en **base euro** et publiés les jours ouvrés uniquement (aucune valeur les week-ends et jours fériés).

Adresse de l'API : <https://api.frankfurter.app> — Documentation : <https://frankfurter.dev>

1. **Taux du jour.** Avec `urllib.request` et `json` (bibliothèque standard), appeler l'URL suivante :
`https://api.frankfurter.app/latest?from=EUR&to=USD`
La réponse a la même forme que le dictionnaire de la partie C.1. Afficher le taux USD et la date. (Remplacer USD par la devise de votre groupe ; le paramètre `from` fixe la devise de base, `to` la devise voulue.)
2. **Série sur une période.** Appeler l'URL couvrant **du 1^{er} janvier 2026 au 1^{er} septembre 2026** (deux dates séparées par `..`) :
`https://api.frankfurter.app/2026-01-01..2026-09-01?from=EUR&to=USD`
Cette fois, la partie `rates` est indexée **par date** (une entrée par jour ouvré). Enregistrer la réponse brute dans `cache/`, puis écrire une version simple en CSV (une ligne par date) dans `donnees/`, en Python de base.

Questions — à rédiger et à commiter

1. Qu'est-ce qu'une API ? Que signifie « envoyer une requête » pour récupérer une donnée ? Qu'est-ce que le format JSON, et en quels types Python se traduit-il ?
2. Pourquoi enregistrer la réponse de l'API dans un fichier (`cache/`) plutôt que de rappeler l'API à chaque exécution ?

Contraintes imposées

Cette séance n'utilise que la **bibliothèque standard** (`urllib`, `json`, `csv`, `datetime`, `pathlib`). Les librairies `pandas`, `numpy` et la librairie tierce `requests` seront découvertes plus tard.

Livrables de la séance (commit + push + PR avant la fin)

Dépôt avec `README.md` (noms du groupe) — `hello_world.py` — `bases_python.py` (types, opérations, conditions, boucles) — script du premier appel API — `cache/` et CSV simple dans `donnees/` — `docs/notes_seance1.md` (réponses aux questions) — **PR de la séance 1** ouverte.